ParallelNTT: Maximizing Performance of Forward and Inverse NTT on FPGA for ML-DSA and ML-KEM

Bardia Taghavi
Florida Atlantic University
Boca Raton, FL, USA
staghavi2024@fau.edu

Reza Azarderakhsh
Florida Atlantic University
Boca Raton, FL, USA
razarderakhsh@fau.edu

Mehran Mozaffari Kermani
University of South Florida
Tampa, FL, USA
Mehran2@usf.edu

Abstract

The rise of quantum computing threatens traditional cryptography, driving the need for post-quantum cryptography (PQC) algorithms like ML-KEM and ML-DSA. These Module-Lattice-Based Cryptography (MLBC) schemes rely on efficient polynomial multiplication, accelerated by the Number Theoretic Transform (NTT) from $O(n^2)$ to $O(n \log_2 n)$ complexity. This paper introduces ParallelNTT, an FPGA-based architecture optimizing forward and inverse NTT for ML-DSA and ML-KEM. By employing a fully parallelized design that eliminates RAM bottlenecks—feeding all coefficients to NTT stages simultaneously—ParallelNTT achieves a groundbreaking $O(\log_2 n)$ latency. Key innovations include taking advantage of the constant geometric structure for simplified routing, K^2-RED modular reduction for efficient arithmetics, and accessing twiddle factor storage shared across forward and inverse NTT, reducing area and boosting speed. Implemented on Xilinx Virtex UltraScale+ and Artix-7 FPGAs, it achieves frequencies up to 400 MHz and latencies as low as 0.015 μs for ML-KEM and 0.017μs for ML-DSA, demonstrating up to 24× improvement in latency over prior FPGA-based implementations while optimizing resource utilization. This scalable, very high-performance solution enhances the deployment of MLBC schemes in high-throughput environments, offering a robust foundation for future PQC systems.

CCS Concepts

• **Security and privacy** → *Hardware security implementation*; **Cryptography**; • **Hardware** → **Reconfigurable logic and FPGAs**.

Keywords

Post-Quantum Cryptography, NTT, FPGA, Hardware Accelerated Arithmetics, High-Performance Computing

ACM Reference Format:
Bardia Taghavi, Reza Azarderakhsh, and Mehran Mozaffari Kermani. 2025. ParallelNTT: Maximizing Performance of Forward and Inverse NTT on FPGA for ML-DSA and ML-KEM. In *Great Lakes Symposium on VLSI 2025 (GLSVLSI '25), June 30–July 02, 2025, New Orleans, LA, USA.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3716368.3735214

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
GLSVLSI '25, New Orleans, LA, USA
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1496-2/25/06
https://doi.org/10.1145/3716368.3735214

1 Introduction

After the long history of traditional cryptography and digital signature schemes like RSA and ECC, the security of these cryptosystems is threatened by Shor's algorithm in 1994, [28], which can be implemented only on a quantum computer in polynomial time, and makes breaking the traditional cryptography feasible. The National Institute of Standards and Technology has initiated a process to standardize Post-Quantum Cryptography (PQC) algorithms, which cannot be broken, even on quantum computers. ML-KEM [5] and ML-DSA [2], two Module-Lattice-Based Cryptography schemes, are among the winners in the NIST post-quantum cryptography standardization process, designed for secure key exchange mechanism and digital signature, respectively. Both cryptosystems, alongside other PQC schemes like FALCON, use several multiplications of polynomials over a ring, which can be implemented using Number Theoretic Transforms. NTT is used to accelerate polynomial multiplication, which is the most computationally intensive operation in these schemes [1].

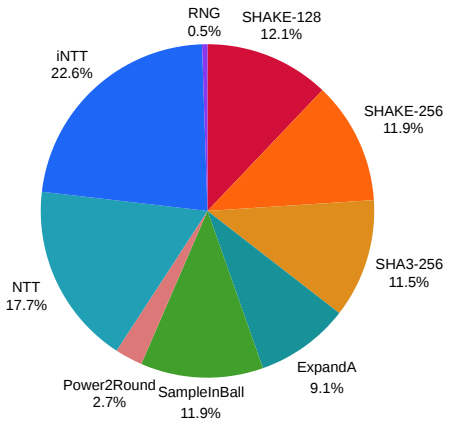


Figure 1: Dynamic Power Consumption of Different Blocks of ML-DSA in Hardware Implementation [26].

The time complexity of polynomial multiplication is $O(n^2)$, but using NTT, the result of the multiplication over a ring can be computed in time complexity of $O(n \log_2 n)$. Although the software implementation of high-performance NTT using CPU or GPU has been extensively studied, the hardware implementation of *extremely fast* NTT on Field-Programmable Gate Arrays (FPGAs) faces some challenges in optimizing performance metrics such as clock cycles, modular reduction, ATP (Area-Time Product), and overall computational efficiency.

The implementation of NTT on FPGA is a crucial part of post-quantum key exchange and digital signature schemes since it consumes more time and power inside the MLBC schemes than any other blocks, and the high-speed NTT computation is vital for MLBC schemes. Specifically, in the implementation of NTT on specific performance-oriented applications such as web servers or low-latency IoT systems, where the speed of key exchange or signature is the only crucial parameter and other factors like resource usage and side-channel attacks (SCA) are not a concern, the NTT computation should be optimized to achieve the highest possible performance and the lowest latency. Figure 1 shows the dynamic power consumption of different blocks of ML-DSA in hardware implementation. As can be seen, the forward and inverse NTT computation consumes the most power in the ML-DSA hardware implementation, and optimization of NTT operation is crucial.

In this paper, we present a parallelized very high-performance FPGA implementation of forward and inverse NTT for polynomial multiplications, specifically for ML-DSA and ML-KEM, which is 24x and 21x faster than the SOTA works. The aim of this paper is to solve the complexities in fully parallelized NTT computation and routing problems.

1.1 Related Work

The primary challenge in high-performance NTT implementation is minimizing the number of clock cycles required for computations. Several studies have proposed innovative architectures to achieve this. For instance, a radix-4-based configurable NTT design has been shown to outperform traditional radix-2 and radix-4 architectures, achieving a 1.5x and 2.3x improvement in speed, respectively [29]. Similarly, a low-latency FPGA implementation of NTT for Kyber achieved state-of-the-art (SOTA) performance, completing NTT/iNTT operations in just 0.4/0.5μs, surpassing existing designs by at least 3.75/3 times [27]. Researchers have explored scalable and parallel optimization techniques to further enhance the performance. By analyzing the algorithm flow of NTT, inverse NTT (iNTT), and pointwise multiplication (PWM), a loosely coupled FPGA structure was designed to support multiple pipelines and various reduction operations [13], and a high-performance result has been obtained. To address the versatility of NTT structures for different lattice-based algorithms, researchers have proposed unified architectures that can support multiple cryptographic schemes. For example, a unified NTT architecture for both CRYSTALS-Kyber and CRYSTALS-Dilithium has been developed, featuring configurable radix-2 butterfly units and a conflict-free memory mapping scheme [16]. In paper [8] a parallelized implementation of NTT on FPGA has been proposed, which achieves a higher performance compared to the state-of-the-art NTT architectures. Although the area consumption of this method is higher than other works, the proposed architecture in this paper lacks the ability to compute the NTT in $O(\log_2 n)$ time complexity, and the latency of the NTT computation is still $O(n)$. Another notable work in this area is the implementation of [27], which achieves very high performance in terms of conversion rate, but the complexity of the architecture limits the frequency to 210 MHz, and it requires 85 clock cycles to compute the NTT. Pipelining the NTT computation is another approach to increase the performance of the NTT computation. In another attempt to improve the performance of the NTT computation, [20] proposed a pipelined NTT architecture optimized for Kyber and Dilithium, achieving significant speedup through dedicated DSP-based modular multiplication and efficient memory access schemes, with high resource usage and very low conversion latency, which is 0.6μs.

GPUs have been used to accelerate NTT computations. For example, an efficient GPU implementation of NTT, inverse NTT, and NTT-based polynomial multiplication operations has improved the performance of these operations by up to 141.95 times compared to CPU-based implementations [23]. In the paper [30], the authors present a GPU-based accelerator designed to enhance polynomial arithmetic operations in Fully Homomorphic Encryption (FHE). By distributing computations across multiple GPUs, their approach achieves up to a 7.66x performance boost compared to single-GPU implementations.

These studies demonstrate the importance of optimizing NTT computation for MLBC schemes and the potential for further improvements in performance and efficiency.

1.2 Contribution

The main contributions of this paper are as follows:

- Although some of the previous works have proposed utilizing constant-geomoetry NTT computation or K^2-RED reduction, fully parallelized NTT computation and the challenges in the implementation and optimization has not been explored. We propose a hardware architecture for forward and inverse NTT to minimize the routing complexity and facilitate the placement of the efficiently parallelized NTT calculation block on FPGA and ASIC. In this architecture, we don't use any RAM modules to store the data, and we feed the elements in the vector of the polynomial to the NTT/iNTT stages simultaneously. As shown in the results, the proposed architecture achieves the best ATP among all implementations of NTT, by computing NTT with the time complexity of $O(\log_2 n)$.
- We provide a novel way to store the twiddle factors on FPGA Read-Only Memory (ROM) to reduce the area consumption and increase the speed of the NTT computation in parallel NTT computation. Using this method, the required twiddle factors for calculating each stage of forward NTT are stored in ROM, and they can be accessed in reverse order in the computation of inverse NTT. As a result, only one storage is required to store the twiddle factors for both forward and inverse NTT computation in a fully parallelized NTT converter.
- We present the required optimizations to reach a higher frequency by reducing the critical path of the forward and inverse NTT computation and increasing the performance of the computation. In this work, we reach a frequency of 400 MHz on Virtex UltraScale+ FPGA and 250 MHz on Xilinx Artix-7 FPGA.
- The details and result of hardware implementation are presented in this paper. The proposed architecture is implemented on Xilinx Virtex UltraScale+ and Artix-7 FPGAs.

2 Background

2.1 Notation

In this paper, we denote the ring of $\mathcal{R}_q = \mathbb{Z}_q[x]/(x^n + 1)$, where q is a prime number and $n = 256$. The polynomial $a = a_0 + a_1 x + \cdots + a_{n-1}x^{n-1} \in \mathcal{R}_q$ is represented as $a = \{a_0, a_1, \ldots, a_{n-1}\}$, $\forall a_i, a_i \in [0, q)$. $\hat{a}$ represents the NTT(a), and $\hat{a} = \{\hat{a}_0, \hat{a}_1, \ldots, \hat{a}_{n-1}\}$.

Although in most of the literatures n-th root of unity is denoted with ω, i.e., $\omega^n \equiv 1 \pmod q$, and $2n$-th root of unity is denoted with ψ, in this paper, we denote the n-th root of unity with ω and the $2n$-th root of unity with ζ, i.e., $\zeta^{2n} \equiv 1 \pmod q$, and $\zeta \equiv \omega^2 \pmod q$. The reason for this is NIST uses ζ for the $2n$-th root of unity in the full NTT definition in ML-DSA specifications [21]. It is worth noting that in ML-KEM specifications which use the incomplete NTT, the n-th root of unity is denoted as ζ [22].

Finally, the forward NTT is denoted as NTT, and the inverse NTT is denoted as iNTT. Throughout this paper, the NTT and iNTT are used to represent the negacyclic NTT and negacyclic iNTT, respectively.

2.2 Number Theoretic Transforms (NTT)

Both ML-KEM and ML-DSA perform the arithmetics over the ring of $\mathcal{R}_q = \mathbb{Z}_q[x]/(x^n + 1)$. Each point on this ring can be represented as a degree $n-1$ polynomial, when each coefficient of this polynomial is in $[0, q)$. Arithmetic operations in these two schemes are addition/subtraction and multiplication. The addition and subtraction need reduction modulo q, which will be discussed in the next part. The result of the multiplication of two degree $n-1$ polynomials is a $2n-2$ degree polynomial; so, a reduction modulo q and a reduction modulo $x^n + 1$ (a series of substitutions of x^n with -1, x^{n+1} with $-x, \ldots, x^{2n-2}$ with $-x^{n-2}$) is necessary to get the correct result. As two alternatives, the computation of the multiplication of two polynomials and the reduction modulo $x^n + 1$ can be done simultaneously using negative-wrap convolution (NWC) or NTT.

A polynomial in $\mathcal{R}_q$ can be represented as a point on an integer lattice, where the lattice bases are in $\mathbb{Z}_q^n$. The multiplication of two polynomials $a = \{a_0, a_1, ..., a_{n-1}\}$ and $b = \{b_0, b_1, ..., b_{n-1}\}$ without reduction is exactly the convolution of a and b. In another approach, the NWC can be used to perform the multiplication and reduction with the irreducible polynomial $x^n + 1$ with a quadratic time complexity $O(n^2)$. As one can use the Fourier transform to convert the convolution to the pointwise multiplication in the complex number domain, the NTT, which is the generalized ring version of the Fourier transform can be used to simplify the polynomial multiplications.

For computing the expression $c = a \cdot b \pmod{\mathcal{R}_q}$ using NTT, three steps are required: (1) Forward NTT of operands a and b to get $\hat{a}$ and $\hat{b}$, (2) Pointwise multiplication of $\hat{a}$ and $\hat{b}$, and (3) Inverse NTT of the result of the pointwise multiplication $\hat{c}$. The forward negacyclic NTT which is used in MLBC schemes is defined as follows:

$$\hat{a_j} = \sum_{i=0}^{n-1} \zeta^i \omega^{ij} \cdot a_i \pmod q = \sum_{i=0}^{n-1} \zeta^{2ij+i} \cdot a_i \pmod q. \quad (1)$$

After transforming the polynomials a and b to $\hat{a}$ and $\hat{b}$ in NTT domain using the forward NTT, the pointwise multiplication of $\hat{a}$ and $\hat{b}$ can be done by multiplying the coefficients of $\hat{a}$ and $\hat{b}$ in the ring of $\mathbb{Z}_q$:

$$\hat{c}_j = \hat{a}_j \circ \hat{b}_j \pmod q. \quad (2)$$

Then, the inverse NTT will be used to convert the result back to the normal domain. The inverse NTT is defined as follows:

$$c_i = n^{-1} \sum_{j=0}^{n-1} \zeta^{-2ij-i} \cdot \hat{c}_j \pmod q. \quad (3)$$

The time complexity of converting from the normal domain to the NTT domain and vice versa using the above equations is $O(n^2)$, and it is still a slow method for multiplication. However, by taking advantage of recursive patterns in Equations (1) and (3), the forward and inverse NTT can be implemented using the Cooley-Tukey (CT) [7] and Gentleman-Sande (GS) [9] butterfly units (BFU), respectively, which are divide-and-conquer algorithms that recursively divide the input into smaller subproblems until the base case is reached. The CT and GS algorithms can be implemented using the butterfly structure, which is a simple structure that can be used to increase the speed of NTT computation. Both algorithms use a step of addition/subtraction and multiplication, and the time complexity of the NTT using CT and GS algorithms is $O(n \log_2 n)$.

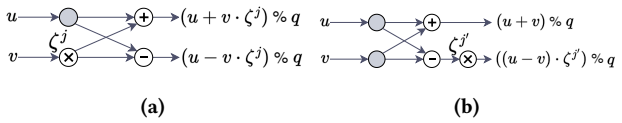


Figure 2: (a) CT and (b) GS BFU used in NTT and iNTT computation, respectively. A modular addition, modular subtraction, and modular multiplication are used in the BFU.

An NTT butterfly structure utilizes CT BFU, and an iNTT makes use of GS BFU by changing the twiddle factors of each pair of BFU and the distance between each pair of BFU inputs. the distance between each pair is not the same for all stages. In fact, it is halved for each stage in NTT computation, and it gets doubled for iNTT computation. This change in the distance may cause problems in parallelized NTT computation.

2.3 Modular Reduction

The modular reduction is a crucial step in NTT computation, which is used to reduce the coefficients of polynomials to the range of $[0, q)$ after each arithmetic operation. The modular reduction can be done using the Barrett reduction, Montgomery reduction, Plantard Reduction [24], K-RED reduction [14] and K^2-RED reduction [4]. The Barrett reduction is a fast method for modular reduction, which is based on the division by a constant. The Montgomery and Plantard reductions are two other fast methods for modular reduction, which need conversion to the Montgomery/Plantard domain, reducing, and converting back to the normal domain. All of these methods have high overhead, time consumption and resource usage. The K-RED reduction and its modified version, K^2-RED reduction, are optimized for the NTT computation. K^2-RED works when the modulo q can be written as $k \cdot 2^m + 1$, which is true for both ML-KEM and ML-DSA. K^2-RED(a,b) computes $a \cdot b \cdot k^2 \pmod q$. Three multiplications and reductions are needed for an NTT multiplier:

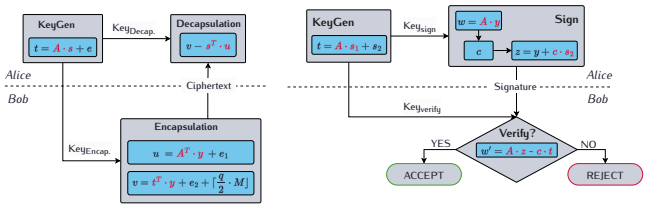


Figure 3: High-Level Overview of ML-KEM (left) and ML-DSA (right). It is shown in the picture that the NTT computation is used in the polynomial multiplication of both schemes multiple times, denoted by dot product.

- **CT BFU**: CT BFU computes $v \cdot \zeta^j \pmod q$. So, we can precompute $k^{-2} \cdot \zeta^j$ and store it in the ROM. As a result, K^2-RED$(v, k^{-2} \cdot \zeta^j)$ can be used to compute $v \cdot \zeta^j \pmod q$.
- **GS BFU**: GS BFU computes $(u - v) \cdot \zeta^j \pmod q$. So, we can precompute $k^{-2} \cdot \zeta^j$ and store it in the ROM. Therefore, K^2-RED$(u - v, k^{-2} \cdot \zeta^j)$ can be used to compute $(u - v) \cdot \zeta^j \pmod q$.
- **Pointwise Multiplication**: The pointwise multiplication of $\hat{a}_i$ and $\hat{b}_i$, i.e., when the second operand is not constant, can be done using two steps of K^2-RED reduction by precomputing $k^{-4} \pmod q$ and then computing K^2-RED$(\hat{a}_i$, K^2-RED$(k^{-4}, \hat{b}_i))$. The result of the pointwise multiplication after reduction is $c_i = \hat{a}_i \cdot \hat{b}_i \pmod q$.

The first two reductions demonstrate the ideality of using K^2-RED reduction for the NTT computation since one of the operands is known. Algorithm 1 shows the method for computing the multiplication of two t-bit numbers using K^2-RED reduction, where $t = \lceil \log_2 q \rceil$ is the number of bits of each number.

Algorithm 1: K^2-RED Reduction

Input: a, b: two t-bit numbers, $q = k \cdot 2^m + 1$: the modulo
Output: c: the result of $a \cdot b \cdot k^{-2} \pmod q$

$p \leftarrow a \times b$;
$cl_1 \leftarrow p[m-1:0]$; $ch_1 \leftarrow p[2 \cdot t : m]$;
$p_1 \leftarrow k \cdot cl_1 - ch_1$;
$cl_2 \leftarrow p_1[m-1:0]$; $ch_2 \leftarrow p_1[2 \cdot t - m : m]$;
$p_2 \leftarrow k \cdot cl_2 - ch_2$;
$c = p_2 < 0 \; ? \; p_2 + q : p_2 \geq q \; ? \; p_2 - q : p_2$;
return c

3 Proposed Architecture

In this section, we present the proposed architecture for the NTT and iNTT computations. The idea of minimizing the latency is to remove the RAM modules and feed the $(\log_2 q \times n)$-bit data to the NTT/iNTT stages simultaneously. The bottleneck in the calculation of NTT is reading the data from RAM one-by-one, two-by-two (in radix-2 BFU), or four-by-four (in radix-4 BFU). In ML-DSA and ML-KEM, $n = 256$, and the wait time to read all coefficients will be at least 64 clock cycles (in radix-4 BFU). To address this problem, we store the vector of polynomials in the flip-flops (FF) and read all coefficients simultaneously, then process all coefficients in radix-2 BFU in one clock cycle for each stage of NTT. This suggests that each NTT/iNTT stage can be done in only one clock cycle, and the latency of the NTT computation will be reduced to $O(\log_2 n)$ asymptotically. In ML-KEM and ML-DSA, $\lceil log_2 n \rceil = 8$, and the latency of the NTT computation will be 8 clock cycles for full NTT in ML-DSA, and 7 clock cycles for incomplete NTT in ML-KEM.

The first problem with this idea is that it complicates the routing and placement of the NTT computation block on FPGA, since the BFUs in different stages receive different elements of input data. In the NTT computation, the distance between each pair of BFUs differs for each stage. To address this issue, we used a constant-geometric structure. For fully parallel computation, in-place computation will complicate the structure and, thus, increase the area usage and reduce the maximum frequency of the design. Therefore, an out-of-place computation architecture for the NTT computation is used here. In this architecture, the distance between each pair of BFUs is the same for all stages. This feature enables us to use the same BFU for all stages of NTT computation, and the routing and placement of the NTT computation block on FPGA will be easier because of reducing the area consumption.

The second problem is the efficient computation of arithmetics inside each BFU. Using Montgomery, Plantard or Barrett reduction inside each BFU will increase the resource usage and reduce the maximum frequency of the design. To address this issue, we use the K^2-RED reduction for the multiplication inside each BFU. The K^2-RED reduction is optimized for the computation inside each BFU since one of the operands is always known, and it is faster than other reduction methods with smaller hardware. Making use of the K^2-RED reduction inside each BFU will help to compute the multiplication in one clock cycle. In addition, we can use this hardware for PWM computation. If some of the multiplications are done using DSP, the LUT usage drops significantly, and since we use a single instruction with multiple data (SIMD), it helps us to reduce the worst negative slack and to utilize a higher frequency.

Each stage has 128 BFUs, thus it needs 128 twiddle factors. The twiddle factor for each stage is precomputed in Algorithm 2. The factor $k^{-2} \pmod q$ is precomputed and multiplied by each twiddle factor to be used later by the BFUs.

The twiddle factors for each stage are stored in the ROM. Therefore, we have 8×128 twiddle factors stored in the ROM for ML-DSA, and 7×128 for ML-KEM, and according to the current stage of computation, the ROM sends out all twiddle factors required for the BFUs of the current stage at once. The current stage is controlled by a Finite State Machine (FSM). An advantage of this method is no separate ROM is needed for storing the twiddle factors for iNTT. The twiddle factors for the iNTT computation are the same as those for the NTT computation, but are accessed in reverse order. Computing the twiddle factors for each stage is finding the *unique* twiddle factors for the stage, and then repeating them until all cells of the twiddle factors array for each stage are filled. The twiddle factors for each stage is called $\text{St}_{\text{stage_num}}\zeta$, which is comprised of $(n/2) \times (\log_2 q)$ elements. To exemplify, for ML-DSA, the twiddle factors for the first stage are $\text{St}_1\zeta = \{k^{-2} \cdot \zeta^{128}, k^{-2} \cdot \zeta^{128}, \ldots, k^{-2} \cdot \zeta^{128}\}$, and the twiddle factors for the second stage are $\text{St}_2\zeta = \{k^{-2} \cdot \zeta^{64}, k^{-2} \cdot \zeta^{192}, k^{-2} \cdot \zeta^{64}, k^{-2} \cdot \zeta^{192}, \ldots, k^{-2} \cdot \zeta^{64}, k^{-2} \cdot \zeta^{192}\}$, and so

Algorithm 2: Precomputing Stageζs for ML-DSA

Input: tfRev: Precomputed array of twiddle factors, stored in bit-reverse order, n: Number of coefficients in the polynomial, k: $k \cdot 2^m + 1$, q: Prime number
Output: StageZs

```
pairNumber ← 0
StageZs ← Empty List
for (pairDist = 256; pairDist ≥ 1; pairDist /= 2) do
    zs ← Empty List
    for (start = 0; start < 256; start += 2·pairDist) do
        pairNumber ← pairNumber +1
        z ← tfRev[pairNumber] ·k^-2 (mod q)
        for (j = start; j < start+pairDist; j++) do
            zs.append(z)
    uniqueZs ← uniqElements(zs)
    zs ← uniqueZs ⋆ (n / len(uniqueZs) / 2)
    StageZs.append(zs)
return StageZs
```

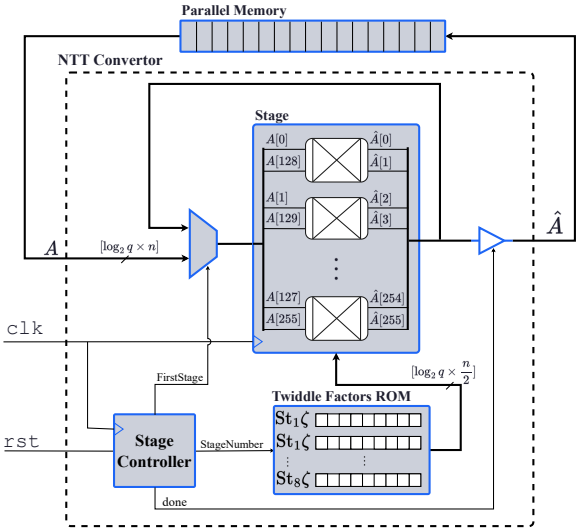


Figure 4: NTT Hardware Architecture

on. In this example, 128 = brv(1), 64 = brv(2), 192 = brv(3), which are the numbers of consecutive BFU, and brv is the BitReverse function.

For the final multiplication in iNTT, since one of the operands is n^{-1} and it is constant, the K^2-RED reduction is used, and $k^{-2} \cdot n^{-1} \pmod q$ is stored in ROM to be used in the final multiplication of iNTT.

3.1 Forward NTT Architecture

Forward NTT block consists of Stage, and ROM. The Stage block receives the coefficients of the polynomial at once, and feeds i-th and $(i + n/2)$-th coefficients to each BFU, $\forall i \in \{0, 1, \cdots, 127\}$. Then, the BFU processes the inputs and sends the result to consecutive outputs. Equation (4) shows the permutation of the output of the BFU.

$$\text{BFU}(\text{Stage}[i, i + \frac{n}{2}]) = \text{Stage}[2i, 2i + 1]. \tag{4}$$

The permutation of the output of the BFU is done by hardwiring the output of the BFU to input of next stage input. This equation is true for all stages. In other words, we can use a single BFU for all stages of NTT computation.

At the same time, the Stage block reads twiddle factors of the current stage from ROM, controlled by FSM to determine the current stage.

Figure 4 shows the hardware architecture of the NTT computation. The input polynomial A is stored in the parallel memory, which is the memory that will be used in a high-performance ML-KEM/ML-DSA implementation, and it will be shared with other components, like KECCAK. The Stage computes the result of the NTT stage at once in one clock. The Twiddle Factors ROM stores the twiddle factors for each stage.

3.2 Inverse NTT Architecture

The iNTT has components like the NTT, with minor modifications. Everything here is the reverse of NTT computation. In essence, the iNTT block consists of the Stage, ROM, and FSM. The Stage block receives the result of the polynomial in NTT domain, and feeds the $2i$-th and $(2i + 1)$-th coefficients to each BFU, $\forall i \in \{0, 1, \cdots, 127\}$. The BFU of iNTT processes the inputs and sends the result to i-th and $(i + n/2)$-th outputs.

$$\text{BFU}(\text{Stage}[2i, 2i + 1]) = \text{Stage}[i, i + \frac{n}{2}]. \tag{5}$$

The Stage block reads twiddle factors of the current stage from ROM, but in the reverse order. That is to say, starts processing the BFUs with the last cell of the last twiddle factors array and ends with the first cell of the first twiddle factors array. This method satisfies the requirement of using only one memory to store the twiddle factors for both forward and inverse parallelized NTT computation.

It is worth mentioning that for ML-KEM, which uses incomplete NTT/iNTT, there are 7 stages for NTT and iNTT computation, and 7 vectors of twiddle factors are stored in the ROM.

4 Evaluation

In this section, we evaluate the performance of the proposed architecture for the NTT computation. We implement the proposed architecture on Xilinx Virtex UltraScale+ xcu250-figd2104 and Artix-7 xc7a100-tcsg324 (NIST recommended platform) FPGAs and evaluate its performance in terms of conversion speed and resource utilization. These FPGA chips have been selected to demonstrate the capability of implementing the proposed design on both large and high-speed (Virtex UltraScale+) and compact and low-budget (Artix-7) FPGA chips. We compare the performance of the proposed architecture with the state-of-the-art NTT architectures on FPGA and GPU. Table 1 shows the summary of the NTT hardware implementations. The proposed architecture achieves a significantly higher performance compared to the state-of-the-art NTT architectures. We implemented the NTT/iNTT with the frequency of 400 MHz ($T = 2.5^{ns}$) on Virtex UltraScale+ and 250 MHz ($T = 4^{ns}$) on Artix-7, and we achieve this frequency by using the simplified architecture, which can implement the 8 stages of NTT with only

Table 1: Summary of NTT Hardware Implementations

Paper	Design Goal	Chip Used	CCs	Freq	Latency (μs)	LUTs	FFs	DSPs	BRAMs	ATP†
[29]	ML-KEM	Virtex-7	306	296	1.03	982	1151	10.5	5	4581
[17]	ML-KEM	Artix-7	933	118	7.91	1243	562	11	3.5	33254
	ML-DSA	Virtex UltraScale+	1074	275	3.91	1178	655	9	9	21722
[10]	ML-KEM	Artix-7	268	216	1.24	904	811	4	2.5	3137
[12]	ML-KEM	Zynq UltraScale+	254	322	3.58	3821	2970	20	5	35463
[20]	ML-KEM	Artix-7	149	239	0.6	9165	9511	0	1	8472
	ML-DSA	Artix-7	64	163	0.39	13804	11019	0	0	7532
[4]	ML-KEM	Artix-7	324	222	1.46	801	717	4	2	3328
[16]	ML-KEM	ZCU+ 102	112	294	0.38	3376	5909	8	5.5	3371
[19]	ML-KEM	Virtex-7	135	256	0.5	6245	1864	24	12	6949
	ML-DSA	Virtex-7	270	175	1.5	1637	2466	20	4.5	13155
[11]	ML-KEM	Artix-7	504	304	1.65	2081	2237	0	0	5279
[13]	ML-DSA	Virtex UltraScale+	113	194	0.58	26520	15171	48	24	29525
[27]	ML-KEM	Artix-7	85	210	0.4	18296	12134	64	0	14353
[15]	ML-DSA	Virtex UltraScale+	155	142.8	1.08	11558	4912	80	40	45375
[8]	ML-DSA	Virtex-7	98	246	0.398	24000	15000	128	0	25273
[18]	(4096, 60)	Virtex-7	785	314	2.5	44420	29415	576	384	279927
[3]	(1024, 30)	Xilinx Versal	660	300	2.2	122650	214782	96	562	366441
[23]	(2048, 30)	NVIDIA Tesla V100	–	–	7	–	–	–	–	–
[30]	(2048, 28)	NVIDIA RTX 3070	–	–	10	–	–	–	–	–
This Work	ML-KEM	Virtex UltraScale+	6	400	0.015	19065	8682	128	0	696
(Very High	ML-DSA	Virtex UltraScale+	7	400	0.017	34595	18565	256	0	1887
Performance	ML-KEM	Artix-7	6	250	0.024	20962	8682	128	0	1160
Design)	ML-DSA	Artix-7	7	250	0.028	41681	18855	240	0	3111

†: ML-KEM: ATP = (LUT + 0.5 · FF + 180 · DSP + 200 · BRAM) · Latency; ML-DSA and others: ATP = (LUT + 0.5 · FF + 250 · DSP + 200 · BRAM) · Latency.

one Stage instance, pipelining the multiplications in DSP, and precomputing the twiddle factors in the ROM to be used in K^2-RED algorithm.

As can be seen, our design is significantly faster than the other implementations, which is ×24 faster than the fastest implementation of ML-KEM and ×21 faster than the fastest implementation of ML-DSA. The ATP of our design illustrates the efficiency of the design, which is the lowest among the other implementations, especially for the ML-KEM. The ATP of the presented architecture shows that the area usage is satisfactory, making it a competitive candidate for high-throughput NTT computation for servers and cloud computing where the speed and throughput are more important than the area usage and no constraints on resources are applied. ATP is computed to fairly compare the designs with different architectures. As in ML-DSA the product two 23-bit numbers are computed and it is larger DSP48Es (which is 18×25), there is a small difference in computing the ATP for ML-DSA and ML-KEM.

Table 2: DSP Utilization Comparison for NTT on Virtex UltraScale+

Mode	Scheme	LUT	FF	DSP	ATP	Power
DSP	ML-KEM	19065	8682	128	696	4.62
NoDSP	ML-KEM	39518	11882	0	681	4.94
DSP	ML-DSA	34595	18565	256	1887	6.44
NoDSP	ML-DSA	101378	22277	0	1969	8.74

Table 2 shows that using DSP in this design helps us to reduce power consumption, especially for ML-DSA, and save the LUTs and FFs considerably. As most computations of MLBC schemes are done in the NTT part and DSP will be rarely used in other parts of NTT, it is acceptable to use DSP for the NTT block. Using DSP will not cause any side issues since the number of DSPs used in the design is less than the number of DSPs available in the FPGA. On the whole, the proposed architecture consumes approximately 1% of the available LUTs, 0.3% of the available FFs, and 6% of the available DSPs on Virtex UltraScale+ FPGA. As a result, a whole MLBC scheme can be implemented easily on a single FPGA chip with a parallelized NTT converter. Table 2 also shows that the normalized ATP equation provides a reasonable basis for comparison.

5 Discussion and Conclusion

By using the novel structure of parallelized NTT stage computation, we can implement the NTT converter with only a single block of Stage since the structures of this block are the same for all stages. As a result, one instance of the Stage is enough for the NTT computation, and this simplification will enhance the routing and placement on FPGAs, leading to higher frequency.

In addition, the latency of the NTT computation is reduced to $O(\log_2 n)$, which is significantly faster than the other hardware and software implementations. For ML-KEM, the presented architecture is 24 times, and for ML-DSA, it is 21 times faster than the state-of-the-art designs. The proposed architecture is also scalable, and it can be used for larger polynomials by simply increasing the number of BFUs in the Stage block and the number of twiddle factors in the ROM. Scalability is crucial for implementing the currently evolving PQC schemes. Moreover, using the proposed hardware architecture in other lattice-based cryptosystems like FALCON and NTRU can be readily adapted by using the presented hardware. The proposed architecture is also flexible, and it can be used for both ML-KEM and ML-DSA by changing the number of stages in the FSM.

Finally, the proposed structure doesn't read the data from the RAM sequentially, and some side-channel attacks that are based on the timing of the data read from the RAM can be prevented, and shuffling-based countermeasures may be unnecessary due to deterministic access patterns [6]. The proposed architecture is also more secure against the cache-based side-channel attacks [25], since the data is stored in the FFs, not in the cache memory, and no pattern exists in accessing the memory. Although the proposed architecture reduces susceptibility to memory-based SCAs, comprehensive countermeasures may be considered for environments where physical access or cache-based leakage is a concern.

Acknowledgments

The authors would like to thank the reviewers for their comments. This work is supported by NSF RINGS 214796.

References

[1] Aikata Aikata, Ahmet Can Mert, Malik Imran, Samuel Pagliarini, and Sujoy Sinha Roy. 2023. KaLi: A Crystal for Post-Quantum Security Using Kyber and Dilithium. *IEEE Transactions on Circuits and Systems I: Regular Papers* 70, 2 (2023), 747–758. doi:10.1109/TCSI.2022.3219555

[2] Shi Bai, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. 2021. CRYSTALS-Dilithium. *Algorithm Specifications and Supporting Documentation* (2021). https://pq-crystals.org/dilithium/data/dilithium-specification-round3-20210208.pdf

[3] Zhenshan Bao, Tianhao Zang, Yiqi Liu, and Wenbo Zhang. 2025. Efficient Number Theoretic Transform accelerator on the versal platform powered by the AI Engine. *Future Generation Computer Systems* 166 (2025), 107728. doi:10.1016/j.future.2025.107728

[4] Mojtaba Bisheh-Niasar, Reza Azarderakhsh, and Mehran Mozaffari-Kermani. 2021. High-Speed NTT-based Polynomial Multiplication Accelerator for Post-Quantum Cryptography. In *2021 IEEE 28th Symposium on Computer Arithmetic (ARITH)*. 94–101. doi:10.1109/ARITH51176.2021.00028

[5] Joppe Bos, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, John M Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. 2018. CRYSTALS-Kyber: a CCA-secure module-lattice-based KEM. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 353–367.

[6] Zhaohui Chen, Yuan Ma, and Jiwu Jing. 2023. Low-Cost Shuffling Countermeasures Against Side-Channel Attacks for NTT-Based Post-Quantum Cryptography. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 1 (2023), 322–326. doi:10.1109/TCAD.2022.3174142

[7] James W. Cooley and John W. Tukey. 1965. An algorithm for the machine calculation of complex Fourier series. *Mathematics of computation* 19, 90 (1965), 297–301.

[8] Yue Geng, Xiao Hu, Minghao Li, and Zhongfeng Wang. 2023. Rethinking Parallel Memory Access Pattern in Number Theoretic Transform Design. *IEEE Transactions on Circuits and Systems II: Express Briefs* 70, 5 (2023), 1689–1693. doi:10.1109/TCSII.2023.3260811

[9] W. Morven Gentleman and G. Sande. 1966. Fast Fourier Transforms: for fun and profit. In *Proceedings of the November 7-10, 1966, Fall Joint Computer Conference (AFIPS Conference Proceedings, Vol. 29)*. Association for Computing Machinery, 563–578. doi:10.1145/1464291.1464352

[10] Yuma Itabashi, Rei Ueno, and Naofumi Homma. 2022. Efficient Modular Polynomial Multiplier for NTT Accelerator of Crystals-Kyber. In *2022 25th Euromicro Conference on Digital System Design (DSD)*. 528–533. doi:10.1109/DSD57027.2022.00076

[11] Khalid Javeed and David Gregg. 2025. Efficient Number Theoretic Transform Architecture for CRYSTALS-Kyber. *IEEE Transactions on Circuits and Systems II: Express Briefs* 72, 1 (2025), 263–267. doi:10.1109/TCSII.2024.3465273

[12] Dur-e-Shahwar Kundi, Jose Maria Bermudo Mera, Pierre-Yves Strub, and Michael Hutter. 2024. High-Performance NTT Hardware Accelerator to Support ML-KEM and ML-DSA. In *Proceedings of the 2024 Workshop on Attacks and Solutions in Hardware Security* (Salt Lake City, UT, USA) *(ASHES '24)*. Association for Computing Machinery, 100–105. doi:10.1145/3689939.3695785

[13] Bin Li, Yunfei Yan, Yuanxin Wei, and Heru Han. 2024. Scalable and Parallel Optimization of the Number Theoretic Transform Based on FPGA. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 32, 2 (2024), 291–304. doi:10.1109/TVLSI.2023.3312423

[14] Patrick Longa and Michael Naehrig. 2016. Speeding up the Number Theoretic Transform for Faster Ideal Lattice-Based Cryptography. In *Cryptology and Network Security: 15th International Conference, CANS 2016, Milan, Italy, November 14-16, 2016, Proceedings (Lecture Notes in Computer Science, Vol. 10052)*. Springer-Verlag, 124–139. doi:10.1007/978-3-319-48965-0_8

[15] Ahmet Malal. 2023. Designing Efficient and Flexible NTT Accelerators. 1617 pages. https://eprint.iacr.org/2023/1617

[16] Suraj Mandal and Debapriya Basu Roy. 2024. KiD: A Hardware Design Framework Targeting Unified NTT Multiplication for CRYSTALS-Kyber and CRYSTALS-Dilithium on FPGA. In *2024 37th International Conference on VLSI Design and 2024 23rd International Conference on Embedded Systems (VLSID)*. IEEE, 455–460. doi:10.1109/VLSID60093.2024.00082

[17] Stefano Di Matteo, Ivan Sarno, and Sergio Saponara. 2024. CRYPHTOR: A Memory-Unified NTT-Based Hardware Accelerator for Post-Quantum CRYSTALS Algorithms. *IEEE Access* 12 (2024), 25501–25511. doi:10.1109/ACCESS.2024.3367109

[18] Xiangchen Meng, Zijun Jiang, and Yangdi Lyu. 2024. HF-NTT: Hazard Free Dataflow Accelerator for Number Theoretic Transform. arXiv:2410.04805 [cs.AR] https://arxiv.org/abs/2410.04805

[19] Jianan Mu, Yi Ren, Wen Wang, Yizhong Hu, Shuai Chen, Chip-Hong Chang, Junfeng Fan, Jing Ye, Yuan Cao, Huawei Li, and Xiaowei Li. 2023. Scalable and Conflict-Free NTT Hardware Accelerator Design: Methodology, Proof, and Implementation. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 5 (2023), 1504–1517. doi:10.1109/TCAD.2022.3205552

[20] Trong-Hung Nguyen, Binh Kieu-Do-Nguyen, Cong-Kha Pham, and Trong-Thuc Hoang. 2024. High-Speed NTT Accelerator for CRYSTAL-Kyber and CRYSTAL-Dilithium. *IEEE Access* 12 (2024), 34918–34930. doi:10.1109/ACCESS.2024.3371581

[21] FIPS NIST. 2024. *Module-lattice-based digital signature standard*. doi:10.6028/nist.fips.204

[22] FIPS NIST. 2024. *Module-lattice-based key-encapsulation mechanism standard*. doi:10.6028/nist.fips.203

[23] Özgün Özerk, Can Elgezen, Ahmet Can Mert, Erdinç Öztürk, and Erkay Savaş. 2022. Efficient number theoretic transform implementation on GPU for homomorphic encryption. *The Journal of Supercomputing* 78, 2 (01 2 2022), 2840–2872. doi:10.1007/s11227-021-03980-5

[24] Thomas Plantard. 2021. Efficient word size modular arithmetic. *IEEE Transactions on Emerging Topics in Computing* 9, 3 (2021), 1506–1518.

[25] Zehua Qiao, Yuejun Liu, Yongbin Zhou, Mingyao Shao, and Shuo Sun. 2023. When NTT Meets SIS: Efficient Side-channel Attacks on Dilithium and Kyber. 1866 pages. https://eprint.iacr.org/2023/1866

[26] Sara Ricci, Lukas Malina, Petr Jedlicka, David Smékal, Jan Hajny, Peter Cíbik, Petr Dzurenda, and Patrik Dobias. 2021. Implementing CRYSTALS-Dilithium Signature Scheme on FPGAs. In *ARES 2021: The 16th International Conference on Availability, Reliability and Security, Vienna, Austria, August 17-20, 2021*. ACM, 1:1–1:11. doi:10.1145/3465481.3465756

[27] Mohamed Saoudi, Akram Kermiche, Omar Hocine Benhaddad, Nadir Guetmi, and Boufeldja Allailou. 2024. Low latency FPGA implementation of NTT for Kyber. *Microprocessors and Microsystems* 107 (2024), 105059. doi:10.1016/j.micpro.2024.105059

[28] Peter W. Shor. 1994. Algorithms for Quantum Computation: Discrete Logarithms and Factoring. In *35th Annual Symposium on Foundations of Computer Science, Santa Fe, New Mexico, USA, 20-22 November 1994*. IEEE Computer Society, 124–134. doi:10.1109/SFCS.1994.365700

[29] Junyan Sun and Xuefei Bai. 2024. A High-Speed Hardware Architecture of an NTT Accelerator for CRYSTALS-Kyber. *Integrated Circuits and Systems* 1, 2 (2024), 92–102. doi:10.23919/ICS.2024.3419562

[30] Zhiwei Wang, Peinan Li, Rui Hou, Zhihao Li, Jiangfeng Cao, XiaoFeng Wang, and Dan Meng. 2023. HE-Booster: An Efficient Polynomial Arithmetic Acceleration on GPUs for Fully Homomorphic Encryption. *IEEE Transactions on Parallel and Distributed Systems* 34, 4 (2023), 1067–1081. doi:10.1109/TPDS.2022.3228628